SQL Foundations

Name: Yalamarthi Viswa Sri Harshith

Unlox – Data Science – Weekly Assignment-3

Section A – Theory

A1. c
A2. b
A3. b
A4. c
A5. b
A6. c
A7. b
A8. c

Section B – Output Prediction

B1.

SELECT COUNT(*) FROM products WHERE category = 'Electronics';

Answer: 8

B2.

SELECT * FROM products WHERE price BETWEEN 1000 AND 3000;

Answer: 3 rows

B3.

SELECT product_name

FROM products

WHERE category = 'Books'

AND price < 400

ORDER BY price DESC

LIMIT 1;

Answer: Rich Dad Poor Dad

B4.

SELECT * FROM products WHERE avg_rating IS NULL;

Answer: 3 rows

B5.

SELECT MAX(price)

FROM products

WHERE category = 'Books';

Answer: ₹599.00

B6.

SELECT category, COUNT(*)

FROM products

WHERE is_active = TRUE

GROUP BY category

HAVING COUNT(*) > 4;

Answer:

Category	Count
Electronics	7
Apparel	6
Home	5

B7.

SELECT product_name,

CASE

WHEN price < 500 THEN 'Budget'

WHEN price < 5000 THEN 'Mid'

ELSE 'Premium'

END AS tier

FROM products

WHERE category='Beauty';

Answer: Returns **4 Beauty products** with two columns:

- product_name
- tier (Budget/Mid/Premium)

B8.

```
SELECT product_name,
COALESCE(avg_rating,0) AS rating
FROM products
WHERE stock_quantity=0;
```

Answer: Returns the **2 out-of-stock products** with ratings. If rating is NULL, it displays **0**.

Section C – Applied SQL

C1

Display all products.

```
SELECT *
FROM products;
```

C2

Books (name and price)

```
SELECT product_name,
       price
FROM products
WHERE category = 'Books';
```

C3

Products above ₹10,000

```
SELECT *
FROM products
WHERE price > 10000
ORDER BY price DESC;
```

C4

Top 5 expensive Electronics

```
SELECT product_name,
       price
```

FROM products

WHERE category = 'Electronics'

ORDER BY price DESC

LIMIT 5;

C5

Electronics or Apparel

SELECT *

FROM products

WHERE category IN ('Electronics', 'Apparel');

C6

Price between ₹500 and ₹2,000

SELECT *

FROM products

WHERE price BETWEEN 500 AND 2000;

C7

Contains "Watch"

SELECT *

FROM products

WHERE product_name LIKE '%Watch%';

C8

Brand starts with S

SELECT *

FROM products

WHERE brand LIKE 'S%';

C9

Unique categories

SELECT DISTINCT category

FROM products;

C10

Total products

```
SELECT COUNT(*) AS total_products
FROM products;
```

C11

Average price of Books

```
SELECT AVG(price) AS average_price
FROM products
WHERE category = 'Books';
```

C12

Maximum and minimum price

```
SELECT MAX(price) AS maximum_price,
    MIN(price) AS minimum_price
FROM products;
```

C13

Products in each category

```
SELECT category,
    COUNT(*) AS product_count
FROM products
GROUP BY category;
```

C14

Total stock per category

```
SELECT category,
    SUM(stock_quantity) AS total_stock
FROM products
GROUP BY category;
```

C15

Average price per category

```
SELECT category,
    AVG(price) AS average_price
FROM products
GROUP BY category
ORDER BY average_price DESC;
```

C16

Brands with more than one product

```
SELECT brand,
    COUNT(*) AS product_count,
    AVG(price) AS average_price
FROM products
GROUP BY brand
HAVING COUNT(*) > 1;
```

C17

Categories with more than 4 active products

```
SELECT category,
    COUNT(*) AS active_products
FROM products
WHERE is_active = TRUE
GROUP BY category
HAVING COUNT(*) > 4;
```

C18

Top 3 expensive products

```
SELECT product_name,
    price
FROM products
```

```
ORDER BY price DESC
LIMIT 3;
```

C19

Categories with average price above ₹2,000

```
SELECT category,
    AVG(price) AS average_price
FROM products
GROUP BY category
HAVING AVG(price) > 2000;
```

C20

Missing ratings

```
SELECT *
FROM products
WHERE avg_rating IS NULL;
```

C21

Display "New Launch" for NULL ratings

```
SELECT product_name,
    COALESCE(CAST(avg_rating AS CHAR), 'New Launch') AS rating
FROM products;
```

C22

Price tier

```
SELECT product_name,
    price,
    CASE
      WHEN price < 1000 THEN 'Budget'
      WHEN price < 10000 THEN 'Mid'
      ELSE 'Premium'
    END AS price_tier
```

```
FROM products;
```

C23

Premium products in each category

```
SELECT category,
       COUNT(*) AS total_products,
       SUM(CASE
               WHEN price >= 10000 THEN 1
               ELSE 0
           END) AS premium_products
FROM products
GROUP BY category;
```

C24

Comprehensive category report

```
SELECT category,
       COUNT(*) AS total_products,
       SUM(CASE
               WHEN is_active = TRUE THEN 1
               ELSE 0
           END) AS active_products,
       AVG(price) AS average_price,
       CASE
           WHEN AVG(price) < 1500 THEN 'Cheap'
           WHEN AVG(price) < 10000 THEN 'Standard'
           ELSE 'Luxury'
       END AS category_tier
FROM products
GROUP BY category
HAVING COUNT(*) >= 3
ORDER BY average_price DESC;
```